

Presenter's notes

Companion to `slides/deck.md`. One section per slide, in deck order. Numbers here match `data/results-full.json`; if you re-run the experiment, refresh the figures with `uv run plot.py --results data/results-full.json` and re-check any figure quoted below.

Rough timing: setup slides 1–5 $\approx$ 4 min · results slides 6–8 $\approx$ 4 min · takeaways $\approx$ 1 min · appendices as needed for questions.

Slide 1 — Should Agents Reason Linearly or as a Tree?

- **Framing sentence to open with:** “We took three papers that all say *let the model think more*, and asked which part of that actually helps.”
 - Four methods, one axis: two of them generate more text (Self-Refine, Tree-of-Thought), one branches, one checks. We want to know which of those is doing the work.
 - Volume: 136 runs, 2 models, 17 tasks, all graded by code rather than by a model.
 - **Keep it short.** The audience needs 30 seconds here, not a minute.
-

Slide 2 — Methods

- Read the table across, not down: **who provides the feedback** is the real difference.
 - Baseline: nobody.
 - Self-Refine: the model itself.
 - Tree-of-Thought: the model itself, but it compares several partial answers instead of one.
 - Agentic-Program-Repair: **the tests**.
 - Self-Refine and APR are both *linear* loops — the only difference is who decides you are done. That makes them a clean pair for comparison.
 - Paper attributions: Self-Refine (Madaan et al. 2023), ToT (Yao et al. 2023), APR (Maddila et al. 2025).
 - **Anticipated question:** “isn’t ToT just beam search?” — Yes, in our implementation: branching with pruning, not full MCTS. ToT’s own Game of 24 experiment uses BFS with a beam of 5, so this is the paper’s practical choice.
-

Slide 3 — Summary: Agentic Program Repair

- Plain-language definition: **when a test breaks, write the code change that makes it pass**. Think Test-Driven Development, automated.
- Read the numbers as a progression — they are the reason we picked APR as our third method:
 - 28.5% agent alone
 - 34.1% add static analysis
 - **43.9% add test execution feedback**

- 61.0% only after repeating the whole loop five times
 - **The key point:** the paper's win is *feedback plus retries*, not a clever search algorithm.
 - **Important for honesty:** the Engineering Agent is an internal Meta system and is **not open-source**, so our implementation comes from the paper's description alone. We kept the part that transfers to small tasks: fresh attempts, judged by tests.
 - **Anticipated question:** "is this MCTS over patches?" — No. The paper does not do tree search, and neither did we.
-

Slide 4 — Our Experiments and task distribution

- Three families, chosen to stress different things: **Game of 24** (search and arithmetic), **Coding** (executable correctness), **Letter counting** (the trap where models confidently say "2 r's in strawberry").
 - **The line to stress:** nothing is graded by a model. Arithmetic, real unit tests in a subprocess, exact counts. That is why the pass/fail numbers can be trusted — an LLM judge could not tell us whether search works.
 - The unit tests are the same feedback signal the APR paper uses, in miniature.
 - Six of the 17 tasks were solved by nobody: five Game of 24 puzzles and `parse_formula`. Worth flagging now, because it shapes every result that follows.
 - Full per-task detail is in `visuals/TABLES.md` if anyone wants to trace a specific number.
-

Slide 5 — Experiment Setup

- Two 7B models from the same era as the papers: `llama2:7b-chat` and `mistral:7b-instruct`.
 - **Say the caveat out loud before anyone asks:** 136 runs is 34 per method. This is directional, not a significance test.
 - Serving details that matter for cost: Ollama on 2 GPUs, 4 parallel slots, 6 tasks in flight.
 - **Cost is reported per solved task**, not per run — total tokens divided by tasks actually solved. A method that burns tokens without solving anything cannot hide behind an average.
 - Full hyperparameters are in Appendix B, if the audience is technical.
-

Slide 6 — Success rate

- How to read the chart: each method has two model bars (blue = Llama-2, orange = Mistral), and each of those splits into three skinny bars in the order **Game of 24 | coding | counting**.
- Headline results: **Agentic-PR 35%**, ToT 29%, Baseline 21%, Self-Refine 18%.
- **The most interesting single result:** ToT goes from 6% on Llama-2 to 53% on Mistral with identical code and prompts. Search amplifies a capable model; it does not rescue a weak one.

- **Game of 24 is a wall:** 1 solve in 48 attempts. The paper's famous 74% is a GPT-4 result — these models cannot do the task at all, so search has nothing to search over.
 - **One counter-intuitive finding:** Self-Refine *never* beats one-shot on any family, and on coding it trails it (2/10 vs 3/10).
 - **If challenged on the math zero:** we scanned every failed math run for a correct answer we had rejected for a missing Answer: line. Zero found — it is a real failure, not a grading bug.
-

Slide 7 — Tokens spent

- Same grouping as the previous chart. Bar = mean over runs, whisker = min to max.
 - **Pattern 1:** Game of 24 costs the most for every method, and it is the family almost nobody solves. Effort and success are not correlated here.
 - **Pattern 2, the headline of the deck:** ToT costs **14,045 tokens per solve on Llama-2 vs 1,524 on Mistral** — a 9× swing for identical code and prompts. Cost is a property of *model plus method*, not of the method alone.
 - **Pattern 3:** Self-Refine is expensive everywhere (3,567 per solve) and still loses to one-shot.
 - **Pattern 4:** Agentic-PR is the cheapest route to accuracy — 1,340 per solve against Baseline's 606, while solving nearly twice as many tasks (12 vs 7).
 - Short Baseline bars are not a formatting artifact: Baseline cannot spend more because it never gets a second attempt.
-

Slide 8 — Wall-clock time

- Per run: **Baseline 2.1s · Agentic-PR 6.1s · Self-Refine 10.0s · Tree-of-Thought 11.2s.**
 - **Interpretation:** time tracks *how many calls* a method makes, not how good it is. Baseline makes 1 call, Self-Refine averages 5.7, ToT 6.9, APR 2.5.
 - **Nice detail:** APR is faster on the stronger model (4.9s vs 7.3s), because a better model passes the tests on an earlier attempt and the loop stops sooner.
 - **State the caveat:** these runs were 6-at-a-time against 4 serving slots, so the absolute numbers include queueing. The *ordering* is the honest part.
 - Useful reframe if asked about serving cost: Self-Refine spends 342 seconds of wall clock across its 34 runs; Baseline spends 73.
-

Slide 9 — Takeaways

- **1. Verification is the win, not the tree.**

Tests turn extra compute into correct answers. More thinking without a check mostly buys longer wrong answers. This is the APR paper's own ablation, reproduced at about 1/50th the scale.

- **2. The methods are not interchangeable.**

Each has a niche: APR owns counting and coding, ToT's best case is a harder task on a stronger model.

- **3. Self-Refine is the weakest link.**

Most expensive, least accurate — worse than one-shot on every family. Its only source of truth is the model's own critique, and a 7B model's critique is only as good as the model.

- **4. Time and cost track the loop, not the result.**
- **5. Cost depends on the model as much as the method**

— the 9× ToT swing again.

- **If asked “so is Tree-of-Thought useless?”** No. It won the single hardest task in the suite (the only Game of 24 solve), came second overall, and its payoff depends on the base model.
 - **If asked what is next:** more runs per task, a stronger base model, and a task set where partial credit is meaningful so pruning has real signal.
-

Appendix A — What APR is in our implementation

- Keep this for a technical audience, because the paper's system is closed-source and someone will ask what we actually built.
 - **What it does:** three independent attempts. Attempt 1 sees the prompt alone; attempts 2 and 3 also see the rejected answer and the instruction to try a *different* approach.
 - **What it is not:** not self-refine, and not a combination. Nothing is carried forward as a partial solution — each attempt is a complete answer, and the tests decide only *whether* to retry.
 - **The honest gap:** we tell the model “the verifier rejected it”, not *which test failed*. The real APR feeds the test output back into the loop, so our signal is weaker than the paper's.
 - **If asked “why not feed the traceback back?”** Our tasks are single-function problems with one-line assertions, so it adds little — but it is a fair criticism and cheap to add.
 - What we dropped from the paper: file reading, stack traces, blame bisection, the 15-action harness. None of it applies to self-contained tasks.
-

Appendix B — Key parameters (1 of 2)

- Run configuration for reproducibility: models, serving stack, concurrency, task counts, run totals.
- **The 450-token output cap** is why some answers look truncated. It never bound on the short counting tasks.
- **The 60-call cap never bound for any method** — ToT averaged 6.9 calls and peaked at 10, so the cap is not what limited it. Worth saying, because “you capped the search” is the obvious objection.

Appendix B — Key parameters (2 of 2)

- Per-method sampling settings. **Temperature differences are deliberate:** the branching methods need diversity to have anything to search over, and the one-shot baseline does not.
- **The verifiers are the part to defend hardest:** unit tests in a subprocess with a 10-second timeout, an exhaustive solver for Game of 24, exact matching for counts. Grading never involves a model.
- **The fairness choice:** `--iters 4` (Self-Refine’s own paper default) and `--candidates 3` give every method a comparable number of attempts, so the cost comparison is not rigged by giving one method more tries.
- **If asked about tuning:** these are paper defaults or round numbers, not swept. A sweep is the obvious next experiment.